can use this function in your program: for example, `multiplicationCurried(3)`, which gives 12 as a result.

NOTE The name `multiplicationCurried` is fictitious. What you call it is your decision.

In Scala, you must explicitly specify when a function can be curried. You do this by separating the individual input variables you want to pass with parentheses. For the multiplication example, you would do so like this:

```
def multiplication(x: Int)(y: Int)
```

You can use the function partially as `multiplication(4)_`. (Note that the underscore is necessary to signal that `y` is not passed.) This gives you back `multiplicationCurried`, which you can use elsewhere in your program.

In the following snippet, a curried function gives the signature you need for the parsing worker:

```
object Manager {
  ...

  def auxCreateRequest(text: String)(replyTo: ActorRef[Worker.Response]):
  ➥Worker.Parse = Worker.Parse(text, replyTo)

  ...
}
```

For example, if you pass a `text` like “text-a” to this function, you will get back the function you need. That function matches the signature of `createRequest: ActorRef[Res] => Req`: